AI-ASSISTED LEGACY APPLICATION MODERNIZATION PLATFORM

A PROJECT REPORT

Submitted by

MUTHUVEER M (310624205166)

in partial fulfillment for the award of the Degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

FULL STACK DEVELOPMENT - 2311CSC503R



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"AI-ASSISTED LEGACY APPLICATION MODERNIZATION PLATFORM"** is the Bonafide work of **"MUTHUVEER M (310624205166)"**, who carried out the project work under my supervision.

SIGNATURE

Dr.N.ANANTHI, M.E.,Ph.D.,

HEAD OF THE DEPARTMENT

Information Technology.,

Easwari Engineering College,

Ramapuram, Chennai-600089.

SIGNATURE

G. Sai Lakshmi,

SUPERVISOR

Information Technology.,

Easwari Engineering College,

Ramapuram, Chennai-600089.

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "AI-ASSISTED LEGACY APPLICATION MODERNIZATION PLATFORM" submitted by Mr. MUTHUVEER M (Reg. No. 310624205166) in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY was conducted on _____ in the Department of Information Technology, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Name :

Designation :

Address :

External Examiner

Name :

Designation :

Address :

Date:

ABSTRACT

Modernizing legacy applications written in COBOL, Visual Basic 6 and ageing Java or C/C++ code bases is a slow, error-prone and expertise-dependent process. Complete manual rewrites are expensive and risky, while simple transpilers treat source code as text and often lose the intent of complex business logic. This project presents LegacyLift, an AI-assisted legacy application modernization platform built on the MERN stack (MongoDB, Express, React and Node.js) that understands legacy code structurally before rewriting it.

The platform runs every upload through an auditable five-stage pipeline: Parse, Analyze, Generate, Validate and Review. Hand-written recursive-descent grammars for COBOL and VB6, and Tree-sitter grammars for Java and C/C++, turn the sources into syntax trees and a language-agnostic intermediate representation. The analyzer builds call graphs, per-procedure control-flow graphs, cyclomatic complexity, read/write sets and a data dictionary, and reports risky constructs such as GO TO, paragraph fall-through and unsafe string handling. A rule-based structural translator then generates Java 21, Python 3.12, TypeScript 5 or C# 12 code with unit tests and build files, preserving fixed-point decimal arithmetic, 1-based indexing and output formats. When an Anthropic API key is configured, a large language model (Claude) refines the draft into idiomatic code, grounded in the source, the analysis and the structural draft, using streamed, schema-validated output.

A validation stage checks every generated file with Tree-sitter, maps each legacy procedure to its modern counterpart, compares control-flow complexity, measures data-item coverage and fixed-point precision, counts generated tests and combines the results into a fidelity score from 0 to 100. Engineers then review, edit, diff, comment on, approve and merge the result and export it as a ZIP archive with a Markdown report. All four bundled samples translate into all four target languages with code that parses, maps every procedure and scores at least 90; the translated COBOL payroll reproduces the exact legacy output, and the code base is covered by 54 automated tests.

ACKNOWLEDGEMENT

I express my sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing me with the opportunity to carry out this project work.

I convey my heartfelt thanks to the **Head of the Department**, Department of **INFORMATION TECHNOLOGY**, for the continuous encouragement and support.

I am extremely thankful to my **Project Guide**, **G. Sai Lakshmi**, Department of **INFORMATION TECHNOLOGY**, for valuable guidance, suggestions, and motivation throughout the completion of this project.

I would also like to extend my gratitude to all the faculty members of the Department of **INFORMATION TECHNOLOGY** for their support.

Finally, I thank my parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	**i**
	ACKNOWLEDGEMENT	**ii**
	LIST OF TABLES	**v**
	LIST OF FIGURES	**vi**
	LIST OF ABBREVIATIONS	**vii**
1	**INTRODUCTION**	**1–3**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	2
	1.5 MOTIVATION	3
2	**RELATED WORK**	**4–6**
	2.1 LITERATURE REVIEW	4
	2.2 EXISTING SYSTEM	4
	2.3 PROPOSED SYSTEM	5
3	**SYSTEM ANALYSIS**	**7–8**
	3.1 SYSTEM REQUIREMENTS	7
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	8
4	**SYSTEM DESIGN**	**9–13**
	4.1 SYSTEM ARCHITECTURE	9
	4.2 MODULE DESCRIPTION	10
	4.3 CLASS DIAGRAM	11
	4.4 SEQUENCE DIAGRAM	12
5	**IMPLEMENTATION**	**14–17**
	5.1 IMPLEMENTATION OVERVIEW	14

CHAPTER NO.	TITLE	PAGE No.
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	14
6	**CONCLUSION & FUTURE WORK**	**18**
	6.1 CONCLUSION	18
	6.2 FUTURE ENHANCEMENTS	18
	APPENDIX	**19–31**
	SCREENSHOTS OF THE PROJECT	**32–38**
	REFERENCES	**39**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Literature survey	4
3.1	Software requirements	8
3.2	Hardware requirements	8

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System architecture of LegacyLift	9
4.2	Class diagram	12
4.3	Sequence diagram of a modernization job	13
A.1	Landing page	32
A.2	Sign-in page	32
A.3	Dashboard with project statistics and bundled samples	33
A.4	Projects list	33
A.5	Creating a new project: upload legacy files and choose the target	34
A.6	Project detail with legacy sources and job history	34
A.7	Job workspace — overview of the pipeline result	35
A.8	Syntax tree explorer linked to the legacy source	35
A.9	Analysis — call graph with entry point and unreachable paragraph	36
A.10	Generated Java code with legacy-to-modern procedure mapping	36
A.11	Validation checks and fidelity score	37
A.12	Human review: file decisions, editor, diff and comments	37
A.13	Modernized codebase after the approved job has been merged	38

LIST OF ABBREVIATIONS

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CFG	Control-Flow Graph
CI	Continuous Integration
COBOL	Common Business-Oriented Language
CRUD	Create, Read, Update and Delete
CSP	Content Security Policy
HTTP	Hypertext Transfer Protocol
IR	Intermediate Representation
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MERN	MongoDB, Express, React and Node.js
ODM	Object Document Mapper
REST	Representational State Transfer
SPA	Single-Page Application
UI	User Interface
VB6	Visual Basic 6

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Enterprises continue to run mission-critical operations such as payroll, billing, inventory and accounting on decades-old code bases that are costly to maintain, difficult to extend and increasingly hard to staff. Much of this code is written in COBOL, Visual Basic 6, early versions of Java or C-style C++, and the people who wrote it are retiring. Complete manual rewrites are expensive and risky, while simple automated transpilers often fail to capture the true intent of complex legacy logic.

LegacyLift is a web platform that modernizes such code through a structure-aware pipeline. It first understands legacy code structurally by parsing it into syntax trees, then reasons about its semantics with static analysis, then generates modern, idiomatic code, validates the result automatically and finally hands it to an engineer for review. The platform is built with the MERN stack: a React 19 single-page application talks to an Express 5 REST API running on Node.js 22, and all users, projects, jobs and review decisions are stored in MongoDB.

Four legacy languages are supported (COBOL, Visual Basic 6, legacy Java and C/C++) and four modern targets can be generated (Java 21, Python 3.12, TypeScript 5 on Node.js and C# 12 on .NET 8), each either as a plain library with unit tests or as a web service in the target ecosystem: Spring Boot, FastAPI, Express or ASP.NET Core minimal APIs.

1.2 PROBLEM DEFINITION

Legacy modernization today relies on manual rewriting by domain experts, rule-based transpilers or pattern-matching migration scripts. These approaches treat source code largely as text rather than as a structured program with well-defined semantics, which causes four recurring problems:

- **High manual effort:** rewriting demands extensive developer time and deep legacy-language expertise that is increasingly scarce.
- **Loss of business logic:** syntax-to-syntax translation mistranslates complex control flow such as COBOL paragraph fall-through, GO TO and PERFORM THRU, and silently changes numeric behaviour such as fixed-point truncation and rounding.

- **No measurable confidence:** existing tools rarely show which legacy procedure became which modern function, or whether any branches were lost on the way.
- **Poor scalability:** each tool usually supports a single language pair, so a mixed code base needs several tools and several review processes.

The problem addressed by this project is therefore to build a platform that translates legacy code into modern languages while preserving its behaviour, explaining its structure, measuring the fidelity of the result and keeping engineers in control of what is finally merged.

1.3 OBJECTIVE

The objectives of the project are:

1. To parse COBOL, Visual Basic 6, legacy Java and C/C++ sources into syntax trees and a single language-agnostic intermediate representation (IR).
2. To analyse the IR and produce call graphs, control-flow graphs, complexity metrics, a data dictionary with suggested modern names and types, and findings about risky constructs and dead code.
3. To generate equivalent code in Java, Python, TypeScript and C#, with unit or characterization tests and build files, using a rule-based structural translator that is optionally refined by a large language model.
4. To validate the generated code automatically and express its fidelity to the legacy program as a score from 0 to 100.
5. To provide a human-in-the-loop review workflow (accept, reject, edit, diff, comment, approve, request changes, merge) and export of the modernized code base.
6. To deliver all of this as a secure, multi-user MERN web application that can be deployed with Docker or on a cloud host.

1.4 SCOPE OF THE PROJECT

The project covers the complete modernization workflow for small and medium-sized business programs:

- **Legacy inputs:** COBOL (fixed, free and variable format, copybooks, level-88 conditions, PERFORM VARYING, EVALUATE), Visual Basic 6 (modules, classes and forms), Java 1.4-style code and C-style C/C++.

- **Targets:** Java 21 (plain or Spring Boot 3), Python 3.12 (plain package or FastAPI), TypeScript 5 (plain module or Express) and C# 12 (.NET 8 class library or ASP.NET Core minimal API).
- **Limits:** a project holds up to 25 files of up to 200 KB each and 2 MB in total; the AI engine accepts projects of up to 3,000 lines; each user can run two jobs at a time.
- **Out of scope:** rare statements such as STRING, UNSTRING, INSPECT, SORT and VB6 GoSub are flagged as `TODO(manual)` markers rather than guessed; COBOL file I/O and numeric overflow are reported for manual review; generated code is never executed on the server, and behavioural comparison happens in the generated characterization tests in the user's own CI.

1.5 MOTIVATION

Large amounts of business-critical software still run on legacy platforms, and every year fewer engineers can read them. Studies of AI-assisted migration suggest that grounding code generation in program structure can reduce manual rewrite effort by more than 60 percent, while keeping translation structure-aware rather than a line-by-line text substitution. The motivation of this project is to show that a full-stack web application can combine classical compiler techniques (parsing, intermediate representations, static analysis) with generative AI, and wrap them in an auditable workflow in which every generated change is measured and reviewed before it is merged.

CHAPTER 2

RELATED WORK

2.1 LITERATURE REVIEW

The design of the platform draws on research in large language models for code, pre-trained code representations, rule-based transpilation and AST-guided refactoring. Table 2.1 summarises the works that most influenced it.

Table 2.1 Literature survey

Author	Year	Description	Pros	Cons
Chen et al.	2021	Trained large language models on code (Codex) to generate functions from context.	Strong code synthesis from pre-trained transformers.	Struggles with long, stateful legacy logic without structural guidance.
Feng et al.	2020	Proposed CodeBERT, a bimodal pre-trained model for code and natural language.	Improved code search, documentation and translation tasks.	Not designed for full program migration or AST-level reasoning.
Rodriguez et al.	2022	Surveyed rule-based transpilers for legacy-to-modern code migration.	Established a baseline for automated syntactic translation.	Fails on ambiguous or undocumented legacy patterns.
Kumar et al.	2023	Combined AST-based program representations with LLMs for guided refactoring.	Improved semantic accuracy over pure prompt-based generation.	Needs robust parsers per legacy language, adding overhead.

Parr's work on ANTLR established grammar-driven parsing as the standard way to build language front ends, and Tree-sitter provides fast, error-tolerant incremental parsers for mainstream languages. Seacord et al. describe modernization as an engineering process in which understanding the existing system comes before transforming it. Together these works motivate the hybrid approach adopted here: static analysis and an intermediate representation provide structure and verifiability, while a language model contributes idiomatic, readable code.

2.2 EXISTING SYSTEM

Existing approaches to legacy modernization fall into three groups:

- **Manual rewriting:** domain experts re-implement the legacy logic in a modern language. The result can be accurate, but the process is slow, costly and depends on scarce experts.
- **Rule-based transpilers:** fixed syntax-to-syntax mapping rules are applied to the source. They are fast, but brittle when code deviates from the expected patterns.
- **Pattern-matching scripts:** regular expressions or templates convert common code blocks, but fail on complex or undocumented logic.

Disadvantages of the existing system

- High manual effort and dependence on legacy-language experts.
- Loss of business logic through mistranslated control flow, risking silent functional regressions.
- Poor scalability across millions of lines and multiple languages.
- Limited semantic understanding: text-based tools cannot reason about program intent, data dependencies or cross-module relationships.

2.3 PROPOSED SYSTEM

The proposed platform first parses legacy source code into an abstract syntax tree to capture its exact structure. A semantic analysis layer then extracts control-flow, data-flow and dependency information and builds a language-agnostic intermediate representation of the program's behaviour. This representation drives two generation engines: a deterministic rule-based structural translator that always runs, and an AI engine based on a large language model that refines the structural draft into idiomatic code when it is enabled. The generated code is validated automatically, scored and handed to an engineer for review before it is merged.

Pipeline: **Parse (AST) → Analyze (Semantics) → Generate (AI / LLM) → Validate (Tests) → Review (Human)**

Advantages of the proposed system

- **Higher translation accuracy:** grounding generation in the AST preserves program structure and business logic more faithfully than text-based translation, including exact fixed-point arithmetic.
- **Multi-language scalability:** a modular parser layer lowers every language into the same IR, so new legacy languages do not require changes to the generators.

- **Reduced manual effort:** structural analysis, code synthesis and test generation are automated; engineers review instead of rewriting.
- **Built-in validation:** every job receives a fidelity score with detailed checks, and AI output is compared with the rule-based baseline.
- **Human control:** nothing is merged until an engineer approves it, and change requests start a linked regeneration that carries the reviewer's feedback.

CHAPTER 3

SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

Functional requirements

FR1. Users can register, sign in and sign out; every project and job is private to its owner.

FR2. Users can create projects by uploading legacy files (with automatic language detection) or from a bundled sample, choose a target language and framework, and edit or delete files.

FR3. Users can start a modernization job with the Auto, AI or rule-based engine, follow its progress stage by stage and cancel it.

FR4. The system shows the syntax tree, call graph, control-flow graphs, procedures, data dictionary and findings of each job.

FR5. The system generates modern source files, tests and build files, and validates them with a fidelity score and individual checks.

FR6. Reviewers can accept or reject files, edit them in the browser, compare them with the generated or legacy version, add line comments, and approve, reject or request changes.

FR7. Approved jobs can be merged into the project's modernized code base and exported as a ZIP archive with a Markdown report.

Non-functional requirements

- **Security:** bcrypt password hashing, JWT sessions in HTTP-only SameSite cookies, Helmet security headers with a content security policy, rate limiting of authentication and job creation, JSON-only write requests and Zod validation of every request body.
- **Reliability:** job state is persisted in MongoDB after every stage; jobs interrupted by a restart are marked as failed so they can be re-run; large job data is stored separately to stay below MongoDB's 16 MB document limit.
- **Performance:** the rule-based pipeline completes a typical sample in well under a second; up to three jobs run concurrently and the client polls a lightweight status endpoint.

- **Usability:** a responsive single-page interface with a live pipeline stepper, interactive graphs and an in-browser code editor.
- **Maintainability:** a modular pipeline with one parser per language, one printer per target, and automated tests for parsers, translators and the REST API.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

Table 3.1 Software requirements

Item	Specification
Operating system	Windows 10/11, Linux or macOS
Runtime	Node.js 22 or above
Database	MongoDB 8 (local, Docker or MongoDB Atlas)
Front end	React 19, React Router 7, Vite 8, Prism.js, dagre, lucide-react
Back end	Express 5, Mongoose 9, Zod 4, jsonwebtoken, bcryptjs, Helmet, express-rate-limit, JSZip
Parsing	Hand-written COBOL and VB6 grammars; web-tree-sitter 0.25 with Java and C/C++ grammars
AI engine	Anthropic SDK with a Claude model (optional, enabled by ANTHROPIC_API_KEY)
Testing	Node.js test runner, Supertest, mongodb-memory-server
Deployment	Docker, Docker Compose, Render blueprint
Browser	Any current version of Chrome, Edge or Firefox

Table 3.2 Hardware requirements

Item	Specification
Processor	2 GHz dual-core or above (multi-core recommended)
RAM	8 GB minimum, 16 GB recommended
Storage	2 GB free space for the application and database (about 800 MB more for the test database binary)
Network	Internet connection for the AI engine and package installation
Monitor	Minimum resolution 1366 × 768, 1920 × 1080 recommended

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

LegacyLift follows a three-tier architecture with a modernization pipeline inside the application tier, as shown in Figure 4.1.

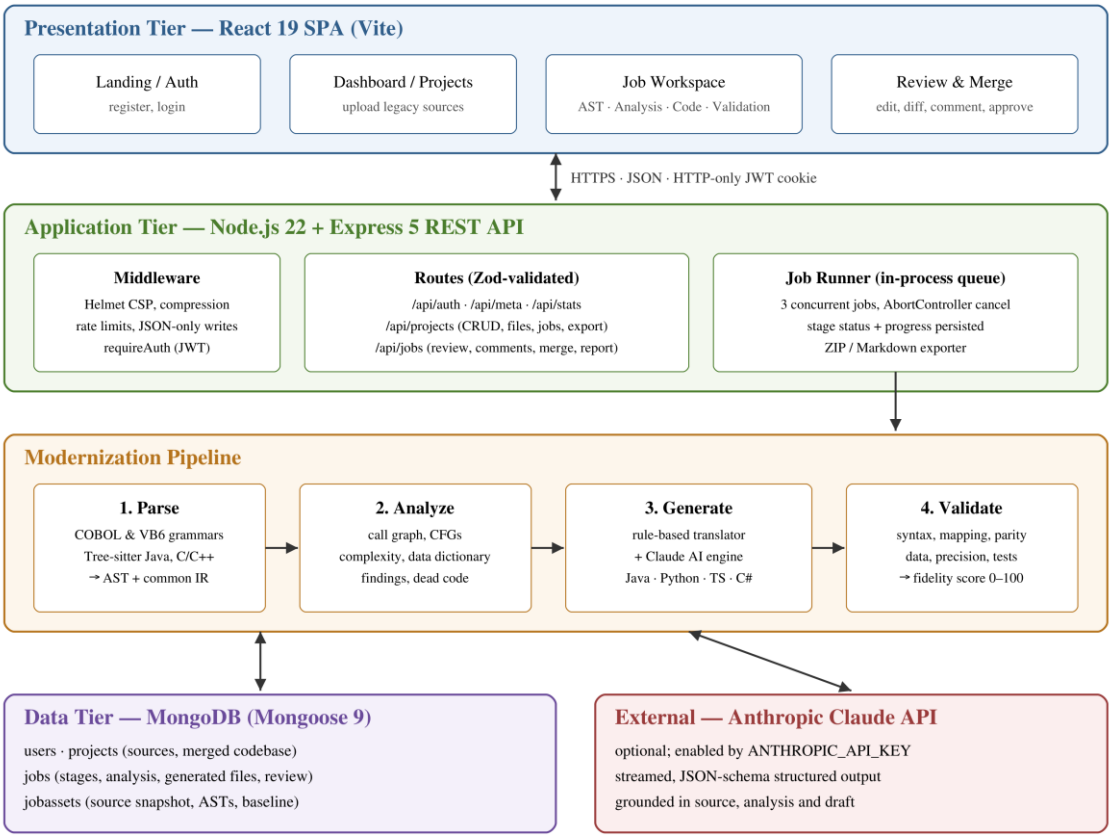


Figure 4.1 System architecture of LegacyLift

The **presentation tier** is a React single-page application built with Vite. It provides the landing and authentication pages, the dashboard, project management, the job workspace with its six tabs and the review screen. It talks to the server only through the JSON REST API and keeps the session in an HTTP-only cookie.

The **application tier** is an Express 5 server running on Node.js 22. Security middleware (Helmet, compression, rate limits and a JSON-only rule for write requests) protects every route; route handlers validate their input with Zod schemas and enforce ownership of every project

and job. In production the same server also serves the built React client, so the whole application runs on a single port.

The **modernization pipeline** runs inside the application tier on an in-process job queue that executes up to three jobs at once. Each job passes through the Parse, Analyze, Generate and Validate stages, and its status, progress messages and results are written to MongoDB after every stage so that the client can follow the job by polling.

The **data tier** is MongoDB, accessed through Mongoose. Users, projects and jobs live in their own collections; large and rarely read job data such as the source snapshot, syntax trees and the rule-based baseline is kept in a separate job asset collection. The optional **Anthropic Claude API** is the only external dependency and is used only by the AI generation engine.

4.2 MODULE DESCRIPTION

Authentication module

Handles registration, login, logout and the current-user endpoint. Passwords are hashed with bcrypt (cost factor 12); a signed JWT is issued in an HTTP-only, SameSite cookie valid for seven days, and the `requireAuth` middleware verifies it on every protected route. Login and registration are rate-limited to 30 attempts per 15 minutes.

Project management module

Creates, lists, updates and deletes projects and their legacy source files. File paths are validated against path traversal, the language of each file is detected from its extension or content, and upload limits are enforced per file and per project. A project can also be created from one of four bundled samples.

Parser module

Converts each source file into a syntax tree and the common IR. COBOL and VB6 are parsed by hand-written recursive-descent grammars; Java and C/C++ are parsed with Tree-sitter grammars compiled to WebAssembly and then lowered into the IR. The IR describes modules, procedures (paragraphs, subs, functions, methods), data items with exact types (for example decimal digits and scale for a COBOL PIC clause) and statements.

Semantic analyzer module

Builds the call graph with entry points and unreachable procedures, a control-flow graph for each procedure, cyclomatic complexity and nesting depth, the data items each

procedure reads and writes, and a data dictionary that suggests modern names and types. It also reports findings such as GO TO, paragraph fall-through, unsafe `strcpy`, raw `Hashtable` or `Vector` collections and missing `Option Explicit`.

Generation module

Contains two engines. The rule-based structural translator always runs: a model layer turns every IR module into a class, and one printer per target language emits source files, characterization tests and build files while preserving COBOL fixed-point semantics, 1-based indexing and output formats. The AI engine, used when an Anthropic API key is configured, sends the legacy source, the analysis and the structural draft to Claude and receives a complete project as schema-validated JSON, with streamed progress.

Validation module

Parses every generated file with Tree-sitter, maps each legacy procedure to a symbol in the generated code, compares cyclomatic complexity to detect lost branches, checks data-item coverage and the use of exact decimal types, counts generated tests and manual-translation markers, and combines these checks into a weighted fidelity score from 0 to 100.

Review, merge and export module

Lets reviewers accept or reject each file, edit it, compare it with the generated or legacy version and add line comments. A job can be approved, rejected or sent back with requested changes, which starts a linked regeneration job carrying the reviewer's feedback. Approved jobs are merged into the project and can be exported as a ZIP archive with a Markdown report.

Job runner module

Queues jobs in memory, runs up to three at a time, supports cancellation through an `AbortController`, records the status of each stage and marks jobs interrupted by a server restart as failed.

4.3 CLASS DIAGRAM

Figure 4.2 shows the persistent classes (Mongoose models) and the printer hierarchy of the rule-based translator. A user owns many projects; a project embeds its source files and has many jobs; a job embeds its generated files and review comments and stores its large data in job assets. Each target language has a printer derived from `BasePrinter`; the C# and TypeScript printers extend the Java printer because they share its brace syntax.

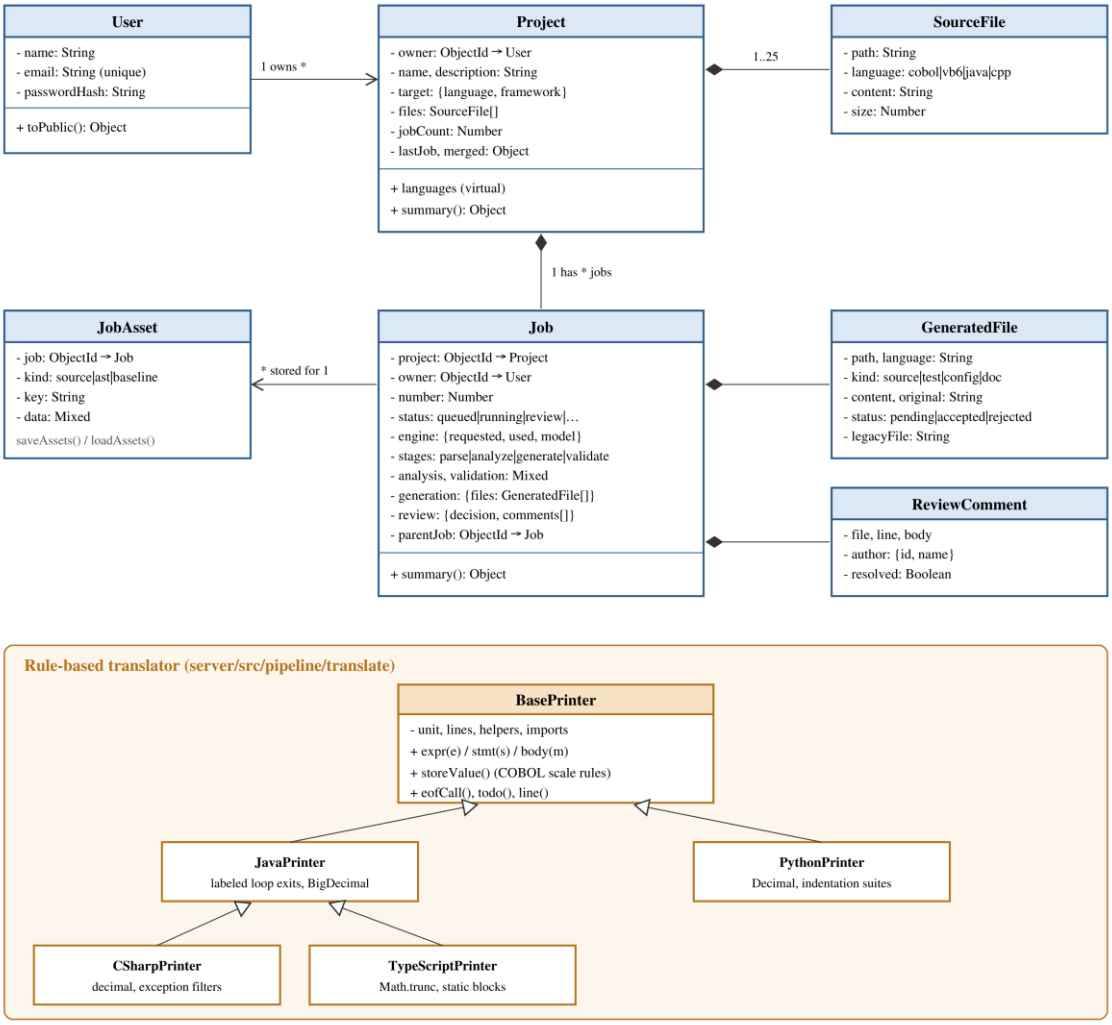


Figure 4.2 Class diagram

4.4 SEQUENCE DIAGRAM

Figure 4.3 shows the life of a modernization job. The client creates the job and receives an immediate 202 response; the job runner executes the pipeline asynchronously while the client polls the status endpoint every 1.5 seconds; finally the engineer reviews, approves and merges the result.

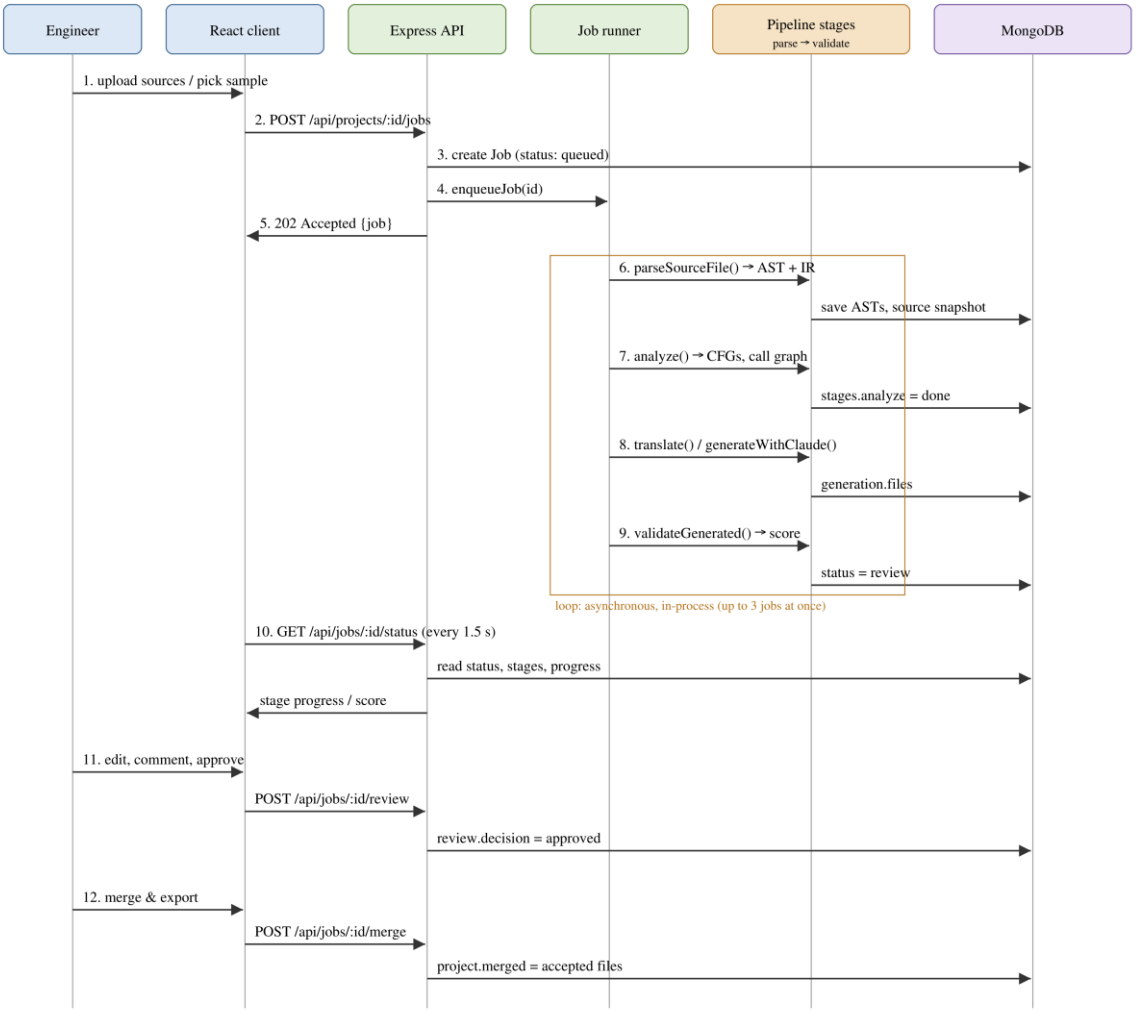


Figure 4.3 Sequence diagram of a modernization job

CHAPTER 5

IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The project is an npm workspace with two packages: `server` (Express API and modernization pipeline) and `client` (React application). The server is written in modern JavaScript modules and starts by connecting to MongoDB, marking interrupted jobs as failed and then listening for requests; it shuts down gracefully on SIGTERM and SIGINT. Configuration comes from environment variables (`MONGODB_URI`, `JWT_SECRET`, `ANTHROPIC_API_KEY` and the upload limits), and production start-up refuses to run without a strong JWT secret.

The pipeline is implemented in `server/src/pipeline`: `parsers/` contains the COBOL and VB6 grammars and the Tree-sitter lowering for Java and C/C++, `analyzer.js` the semantic analysis, `translate/` the rule-based translator, `generator.js` the Claude engine, `validator.js` the validation stage and `runner.js` the job queue. The client uses React Context for authentication, metadata and notifications, React Router for navigation, Prism for syntax highlighting, the `diff` package for comparisons and dagre for graph layout.

The application is covered by 54 automated tests run with the Node.js test runner: parser and analyzer tests, translation of all sixteen sample-and-target combinations, an end-to-end run of the translated COBOL payroll in Python that checks the exact legacy output, regression tests for translator edge cases, the Claude integration against a mock of the streaming API, and the REST API (authentication, access control, the review and merge workflow and limits) against an in-memory MongoDB.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

Choosing the generation engine

`chooseEngine` decides which engine generates a job. The rule-based translator is used when it is requested, when no API key is configured or when the project is larger than the AI limit; an explicit request for the AI engine that cannot be honoured fails with a clear message.

```
export function chooseEngine(requested, totalLines) {
  if (requested === 'structural') return { used: 'structural', reason: 'Rule-based
structural translator selected' };
```

```
  if (!config.ai.enabled) {
    if (requested === 'claude') throw new GenerationError('The AI engine is not
configured on this server (set ANTHROPIC_API_KEY).');
    return { used: 'structural', reason: 'AI engine not configured; used the rule-
based structural translator' };
  }
  if (totalLines > config.limits.maxAiSourceLines) {
    if (requested === 'claude') throw new GenerationError(`The project has
${totalLines} lines; the AI engine accepts up to ${config.limits.maxAiSourceLines}.
Split it into smaller projects.`);
    return { used: 'structural', reason: `Project exceeds
${config.limits.maxAiSourceLines} lines; used the rule-based structural translator`
};
  }
  return { used: 'claude', reason: `AI engine (${config.ai.model}) grounded in the
AST analysis` };
}
```

Running the pipeline

`runJob` executes the four stages in order. After each stage it writes the stage status, a progress message and the results to the job document, and it checks for cancellation between stages. The excerpt below shows the Parse and Analyze stages.

```
// ---- Parse ----
await setStage(jobId, 'parse', { status: 'running', startedAt: new Date() });
const files = project.files.map((f) => ({ path: f.path, language: f.language,
content: f.content }));
const parseTargets = files.filter((f) => !isCopybook(f) || files.every(isCopybook));
const parsed = [];
for (const f of parseTargets) {
  parsed.push(await parseSourceFile(f, files));
  checkCancel();
}
const diagnostics = parsed.reduce((a, p) => a + p.diagnostics.length, 0);
await saveAssets(jobId, 'source', files.map((f) => ({ key: f.path, data: f })));
await saveAssets(
  jobId,
  'ast',
  parsed.map((p) => ({ key: p.file, data: { file: p.file, kind: p.ast.kind, tree:
p.ast.tree, nodeCount: p.ast.nodeCount, truncated: p.ast.truncated } })),
);
await Job.updateOne(
  { _id: jobId },
  {
    $set: {
      parse: {
        files: parsed.map((p) => ({ file: p.file, language: p.language, lines:
p.lines, parser: p.parser, ms: p.ms, nodeCount: p.ast.nodeCount, diagnostics:
p.diagnostics, modules: p.modules.map((m) => ({ name: m.name, kind: m.kind,
procedures: m.procedures.length, dataItems: m.dataItems.length, entry: m.entry }))
})),
        copybooks: files.filter(isCopybook).map((f) => f.path),
      },
      'stages.parse.status': 'done',
      'stages.parse.finishedAt': new Date(),
      'stages.parse.message': `${parsed.length} file(s), ${parsed.reduce((a, p) => a
+ p.modules.length, 0)} module(s), ${diagnostics} diagnostic(s)`,
      progress: { stage: 'analyze', message: 'Building control-flow graphs and data
dictionary' },
    },
  },
);
if (!parsed.some((p) => p.modules.some((m) => m.procedures.length))) throw new
GenerationError('No procedures were found in the uploaded sources; check the language
of each file.');
```

```
// ---- Analyze ----
stage = 'analyze';
await setStage(jobId, 'analyze', { status: 'running', startedAt: new Date() });
const analysis = analyze(parsed);
await Job.updateOne(
  { _id: jobId },
  {
    $set: {
      analysis,
      'stages.analyze.status': 'done',
      'stages.analyze.finishedAt': new Date(),
      'stages.analyze.message': `${analysis.summary.procedures} procedure(s), max
complexity ${analysis.summary.maxComplexity}, ${analysis.findings.length}
finding(s)`,
      progress: { stage: 'generate', message: 'Preparing code generation' },
    },
  },
);
checkCancel();
```

Preserving COBOL storage semantics

`storeValue` reproduces how COBOL stores a computed value into a receiving field: the result is truncated to the field's scale, or rounded half-up when the statement says ROUNDED. Every printer emits a small helper (`scale` in Java and TypeScript, `Scale` in C#, `_scale` in Python) that implements this rule with an exact decimal type.

```
storeValue(s, targetType) {
  const legacyT = targetType;
  if (this.src === 'cobol' && legacyT?.t === 'decimal' && legacyT.scale != null) {
    const v = this.coerce(s.value, DEC);
    if (s.value.k === 'num') {
      const decimals = (s.value.v.split('.')[1] || '').length;
      if (decimals <= legacyT.scale && !s.round) return this.numLit(s.value.v, DEC);
    }
    return this.scaleCall(v, legacyT.scale, Boolean(s.round));
  }
  if (this.src === 'cobol' && (legacyT?.t === 'int' || legacyT?.t === 'long') &&
this.typeOf(s.value).t === 'decimal') {
    return this.toIntFromDecimal(this.scaleCall(this.coerce(s.value, DEC), 0,
Boolean(s.round)));
  }
  return this.coerce(s.value, targetType || UNK);
}
```

Computing the fidelity score

The validation stage combines its checks into one weighted score: syntax 25 %, procedure mapping 25 %, control-flow parity 20 %, data coverage 10 %, generated tests 15 % and remaining manual markers 5 %.

```
const score = Math.round(
  100 * (0.25 * syntaxScore + 0.25 * procCoverage + 0.2 * parityScore + 0.1 *
dataCoverage + 0.15 * testScore + 0.05 * (todoMarkers === 0 ? 1 : todoMarkers < 5 ?
0.5 : 0)),
);
```

Calling the AI engine

`generateWithClaude` streams a request to the Claude Messages API with adaptive thinking and a JSON schema for the output, reports progress while the model writes, and converts API errors into clear, user-facing messages. The returned JSON is validated again with Zod before it is used.

```
const stream = getClient().beta.messages.stream(
  {
    model: config.ai.model,
    max_tokens: config.ai.maxTokens,
    ...(config.ai.fallbacks ? { betas: ['server-side-fallback-2026-07-01'],
fallbacks: 'default' } : {}),
    thinking: { type: 'adaptive', display: 'summarized' },
    output_config: { effort: config.ai.effort, format: { type: 'json_schema', schema:
OUTPUT_SCHEMA } },
    system,
    messages,
  },
  { signal },
);
stream.on('text', (delta) => {
  text += delta;
  emit();
});
stream.on('thinking', (delta) => {
  thinking += delta;
  emit();
});
message = await stream.finalMessage();
```

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 CONCLUSION

This project designed and implemented LegacyLift, an AI-assisted legacy application modernization platform built on the MERN stack. By parsing legacy code into syntax trees and a common intermediate representation before generating anything, the platform turns modernization from text substitution into a structure- and semantics-aware process. The rule-based translator preserves behaviour exactly, including COBOL fixed-point arithmetic, while the optional AI engine improves readability without losing that grounding.

Every job is measured: the validation stage shows which legacy procedure became which modern symbol, whether any branches were lost and how well data and precision were preserved, and the review workflow keeps an engineer in control of what is merged. All bundled samples translate into all four target languages with code that parses, maps every procedure and scores at least 90 out of 100, and the translated COBOL payroll reproduces the legacy program's output exactly.

6.2 FUTURE ENHANCEMENTS

- Move the job runner into a separate worker process with a persistent queue so that the platform can scale out horizontally.
- Execute the generated characterization tests in an isolated sandbox and compare their output with the legacy program automatically.
- Support further legacy languages such as PL/I, RPG and Fortran through new parser front ends.
- Translate the constructs that are currently flagged for manual work, such as COBOL STRING, UNSTRING and INSPECT, file I/O and ON SIZE ERROR overflow checks.
- Add team workspaces with roles, so that authors and reviewers can be different people, and integrate with Git hosting and CI pipelines.
- Map multiple exception handlers precisely for the Python and TypeScript targets.

APPENDIX

SOURCE CODE:

1. Project Structure

Project Folder Structure

```
MERN_stack_project/
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   ├── AstTree.jsx
│   │   │   ├── CodeEditor.jsx
│   │   │   ├── CodeView.jsx
│   │   │   ├── DiffView.jsx
│   │   │   ├── ErrorBoundary.jsx
│   │   │   ├── FileDropzone.jsx
│   │   │   ├── Graph.jsx
│   │   │   ├── Layout.jsx
│   │   │   ├── PipelineStepper.jsx
│   │   │   ├── ScoreRing.jsx
│   │   │   └── ui.jsx
│   │   ├── lib/
│   │   │   ├── actions.js
│   │   │   ├── auth.jsx
│   │   │   ├── format.js
│   │   │   ├── meta.jsx
│   │   │   ├── prism.js
│   │   │   └── toast.jsx
│   │   ├── pages/
│   │   │   ├── job/
│   │   │   │   ├── AnalysisTab.jsx
│   │   │   │   ├── AstTab.jsx
│   │   │   │   ├── CodeTab.jsx
│   │   │   │   ├── OverviewTab.jsx
│   │   │   │   ├── ReviewTab.jsx
│   │   │   │   └── ValidationTab.jsx
│   │   │   ├── Auth.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── JobView.jsx
│   │   │   ├── Landing.jsx
│   │   │   ├── NewProject.jsx
│   │   │   ├── NotFound.jsx
│   │   │   ├── ProjectDetail.jsx
│   │   │   └── Projects.jsx
│   │   ├── api.js
│   │   ├── App.jsx
│   │   ├── main.jsx
│   │   └── styles.css
│   ├── index.html
│   ├── package.json
│   └── vite.config.js
├── docs/
│   └── AI_Legacy_Modernization_Platform.md
├── server/
│   ├── scripts/
│   │   └── dev-mongo.js
│   ├── src/
│   │   ├── middleware/
│   │   │   ├── auth.js
│   │   │   └── errors.js
│   │   ├── models/
│   │   │   ├── Job.js
│   │   │   ├── JobAsset.js
│   │   │   ├── Project.js
│   │   │   └── User.js
```

```
│   │   ├── pipeline/
│   │   │   ├── parsers/
│   │   │   │   ├── cfamily.js
│   │   │   │   ├── cobol.js
│   │   │   │   ├── index.js
│   │   │   │   └── vb6.js
│   │   │   ├── translate/
│   │   │   │   ├── index.js
│   │   │   │   ├── model.js
│   │   │   │   └── printers.js
│   │   │   ├── analyzer.js
│   │   │   ├── catalog.js
│   │   │   ├── generator.js
│   │   │   ├── ir.js
│   │   │   ├── runner.js
│   │   │   ├── treesitter.js
│   │   │   └── validator.js
│   │   ├── routes/
│   │   │   ├── auth.js
│   │   │   ├── jobs.js
│   │   │   ├── meta.js
│   │   │   └── projects.js
│   │   ├── samples/
│   │   │   ├── billing.cpp
│   │   │   ├── index.js
│   │   │   ├── InventoryModule.bas
│   │   │   ├── LegacyAccountService.java
│   │   │   └── PAYROLL.cbl
│   │   ├── services/
│   │   │   └── exporter.js
│   │   ├── app.js
│   │   ├── config.js
│   │   ├── db.js
│   │   └── index.js
│   ├── test/
│   │   ├── api.test.js
│   │   ├── generator.test.js
│   │   ├── pipeline.test.js
│   │   └── translator-regressions.test.js
│   └── package.json
├── .dockerignore
├── .env.example
├── .gitignore
├── docker-compose.yml
├── Dockerfile
├── package.json
├── README.md
└── render.yaml
```

2. Backend Package Installation

```
# install dependencies for the server and client workspaces
npm install

# create the configuration file, then set JWT_SECRET (and optionally
ANTHROPIC_API_KEY)
cp .env.example .env

# terminal 1: local MongoDB on 127.0.0.1:27017
npm run dev:db

# terminal 2: API on :5000 and the Vite dev server on :5173
npm run dev

# production build: Express serves the built client on one port
npm run build
npm start

# automated tests
npm test
```

3. Database Connection

server/src/db.js

```
import mongoose from 'mongoose';

export async function connectDb(uri) {
  mongoose.set('strictQuery', true);
  await mongoose.connect(uri, { serverSelectionTimeoutMS: 10000 });
  return mongoose.connection;
}

export async function disconnectDb() {
  await mongoose.disconnect();
}
```

4. Environment Variables

.env.example

```
# Copy to .env for local development. In production, set these on your host.

NODE_ENV=development
PORT=5000

# MongoDB connection string (local, Docker, or MongoDB Atlas)
MONGODB_URI=mongodb://127.0.0.1:27017/legacy_modernization

# Required in production: a long random string used to sign login tokens
JWT_SECRET=change-me-to-a-long-random-string

# Optional: enables the AI code generation engine (Claude). Without it the
# platform uses its rule-based structural translator for the Generate stage.
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=claude-opus-5
# low | medium | high | xhigh | max
ANTHROPIC_EFFORT=high
# Server-side refusal fallback (on by default); set to "off" if your model or
platform rejects it
ANTHROPIC_FALLBACKS=default

# Set to 1 when running behind a reverse proxy / load balancer (Render, Railway,
Nginx)
TRUST_PROXY=0

# Limits
MAX_PROJECT_KB=2048
MAX_FILE_KB=200
MAX_FILES_PER_PROJECT=25
MAX_AI_SOURCE_LINES=3000
MAX_CONCURRENT_JOBS_PER_USER=2
```

5. User Model

server/src/models/User.js

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true, maxlength: 80 },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true,
maxlength: 254 },
```

```
    passwordHash: { type: String, required: true, select: false },
  },
  { timestamps: true },
);

userSchema.methods.toPublic = function toPublic() {
  return { id: this._id.toString(), name: this.name, email: this.email, createdAt:
this.createdAt };
};

export const User = mongoose.model('User', userSchema);
```

6. Password Hashing and Registration

server/src/routes/auth.js

```
const RegisterSchema = z.object({
  name: z.string().trim().min(2, 'Name must be at least 2 characters').max(80),
  email: z.string().trim().toLowerCase().pipe(z.email('Enter a valid email
address')),
  password: z.string().min(8, 'Password must be at least 8 characters').max(128),
});

router.post('/register', authLimiter, async (req, res) => {
  const body = parseBody(RegisterSchema, req.body);
  if (await User.exists({ email: body.email })) throw new HttpError(409, 'An account
with this email already exists');
  const user = await User.create({ name: body.name, email: body.email, passwordHash:
await bcrypt.hash(body.password, 12) });
  issueSession(res, user);
  res.status(201).json({ user: user.toPublic() });
});
```

7. Login and JWT Generation

server/src/routes/auth.js

```
router.post('/login', authLimiter, async (req, res) => {
  const body = parseBody(LoginSchema, req.body);
  const user = await User.findOne({ email: body.email }).select('+passwordHash');
  if (!user || !(await bcrypt.compare(body.password, user.passwordHash))) throw new
HttpError(401, 'Incorrect email or password');
  issueSession(res, user);
  res.json({ user: user.toPublic() });
});
```

server/src/middleware/auth.js

```
export function issueSession(res, user) {
  const token = jwt.sign({ sub: user._id.toString(), name: user.name },
config.jwtSecret, { expiresIn: config.tokenTtlSeconds });
  res.cookie(AUTH_COOKIE, token, {
    httpOnly: true,
    sameSite: 'lax',
    secure: config.isProd,
    maxAge: config.tokenTtlSeconds * 1000,
    path: '/',
  });
  return token;
}
```

8. Authentication Middleware

server/src/middleware/auth.js

```
export function requireAuth(req, res, next) {
  const header = req.headers.authorization;
  const token = req.cookies?.[AUTH_COOKIE] || (header?.startsWith('Bearer ') ?
header.slice(7) : null);
  if (!token) return next(new HttpError(401, 'Sign in to continue'));
  try {
    const payload = jwt.verify(token, config.jwtSecret);
    req.user = { id: payload.sub, name: payload.name };
    return next();
  } catch {
    return next(new HttpError(401, 'Your session has expired; sign in again'));
  }
}
```

9. Create API

server/src/routes/projects.js — POST /api/projects

```
router.post('/', async (req, res) => {
  const body = parseBody(CreateSchema, req.body);
  let files = body.files || [];
  let target = body.target;
  if (body.sampleId) {
    const sample = loadSample(body.sampleId);
    if (!sample) throw new HttpError(400, 'Unknown sample');
    files = [...files, { path: sample.file, content: sample.content, language:
sample.language }];
    target ||= sample.target;
  }
  if (!target) throw new HttpError(400, 'target: choose a target language and
framework');
  const prepared = files.map(prepareFile);
  assertProjectSize(prepared);
  const paths = new Set();
  for (const f of prepared) {
    if (paths.has(f.path)) throw new HttpError(400, `Duplicate file path ${f.path}`);
    paths.add(f.path);
  }
  const project = await Project.create({ owner: req.user.id, name: body.name,
description: body.description, target, files: prepared });
  res.status(201).json({ project: project.summary() });
});
```

10. Read API

server/src/routes/projects.js — GET /api/projects, GET /api/projects/:id

```
router.get('/', async (req, res) => {
  const projects = await Project.find({ owner: req.user.id }).sort({ updatedAt: -1
}).select('-merged.files.content');
  res.json({ projects: projects.map((p) => p.summary()) });
});

router.get('/:id', async (req, res) => {
  const project = await loadProject(req);
  const jobs = await Job.find({ project: project._id }).sort({ number: -1
}).select('number status engine target stages validation.score error parentJob
review.decision generation.files.path createdAt finishedAt');
  res.json({
    project: { ...project.summary(), files: project.files.map(fileView) },
    jobs: jobs.map((j) => j.summary()),
```

```
  });
});
```

11. Update

server/src/routes/projects.js — PATCH /api/projects/:id

```
router.patch('/:id', async (req, res) => {
  const project = await loadProject(req);
  const body = parseBody(UpdateSchema, req.body);
  Object.assign(project, body);
  await project.save();
  res.json({ project: project.summary() });
});
```

12. Delete

server/src/routes/projects.js — DELETE /api/projects/:id

```
router.delete('/:id', async (req, res) => {
  const project = await loadProject(req);
  const jobs = await Job.find({ project: project._id }).select('_id status');
  jobs.filter((j) => ['queued', 'running'].includes(j.status)).forEach((j) =>
cancelJob(j._id));
  await Job.deleteMany({ project: project._id });
  await deleteAssetsFor(jobs.map((j) => j._id));
  await project.deleteOne();
  res.json({ ok: true });
});
```

13. Express Routes

server/src/app.js

```
import fs from 'node:fs';
import path from 'node:path';
import compression from 'compression';
import cookieParser from 'cookie-parser';
import express from 'express';
import helmet from 'helmet';
import morgan from 'morgan';
import { config } from './config.js';
import { HttpError, errorHandler, notFound } from './middleware/errors.js';
import authRoutes from './routes/auth.js';
import jobRoutes from './routes/jobs.js';
import metaRoutes from './routes/meta.js';
import projectRoutes from './routes/projects.js';

export function createApp() {
  const app = express();
  app.disable('x-powered-by');
  if (config.trustProxy) app.set('trust proxy', config.trustProxy);

  app.use(
    helmet({
      contentSecurityPolicy: {
        directives: {
          'script-src': ["'self'"],
          'img-src': ["'self'", 'data:'],
          'connect-src': ["'self'"],
        },
      },
    }),
```

```
  );
  app.use(compression());
  if (!config.isTest) app.use(morgan(config.isProd ? 'combined' : 'dev'));
  app.use(express.json({ limit: '4mb' }));
  app.use(cookieParser());

  // Defence in depth on top of SameSite cookies: state-changing calls must be JSON,
  // which a cross-site form cannot send.
  app.use('/api', (req, res, next) => {
    if (['POST', 'PUT', 'PATCH', 'DELETE'].includes(req.method) &&
!req.is('application/json')) {
      return next(new HttpError(415, 'Send API requests as application/json'));
    }
    return next();
  });

  app.use('/api', metaRoutes);
  app.use('/api/auth', authRoutes);
  app.use('/api/projects', projectRoutes);
  app.use('/api/jobs', jobRoutes);
  app.use('/api', notFound);

  const indexHtml = path.join(config.clientDist, 'index.html');
  if (fs.existsSync(indexHtml)) {
    app.use(
      express.static(config.clientDist, {
        index: false,
        setHeaders: (res, filePath) => {
          if (filePath.includes(`${path.sep}assets${path.sep}`))
res.setHeader('Cache-Control', 'public, max-age=31536000, immutable');
        },
      }),
    );
    app.get(/^(?!\/api(\/|$)).*/, (req, res) => res.sendFile(indexHtml));
  }

  app.use(errorHandler);
  return app;
}
```

14. Main Server File

server/src/index.js

```
import { createApp } from './app.js';
import { config } from './config.js';
import { connectDb, disconnectDb } from './db.js';
import { recoverInterruptedJobs } from './pipeline/runner.js';

async function main() {
  await connectDb(config.mongoUri);
  const recovered = await recoverInterruptedJobs();
  if (recovered) console.log(`[startup] marked ${recovered} interrupted job(s) as
failed`);
  if (config.jwtSecretIsEphemeral) console.warn('[startup] JWT_SECRET is not set;
using a random secret (sessions reset on restart)');
  console.log(`[startup] AI engine: ${config.ai.enabled ? `${config.ai.model} (effort
${config.ai.effort})` : 'not configured; the rule-based translator will be used'}`);

  const app = createApp();
  const server = app.listen(config.port, () => {
    console.log(`[startup] Legacy Modernization Platform listening on
http://localhost:${config.port} (${config.env})`);
  });

  const shutdown = (signal) => {
    console.log(`[shutdown] ${signal} received`);
    server.close(async () => {
      await disconnectDb().catch(() => {});
      process.exit(0);
```

```
    });
    setTimeout(() => process.exit(1), 10000).unref();
  };
  process.on('SIGTERM', () => shutdown('SIGTERM'));
  process.on('SIGINT', () => shutdown('SIGINT'));
}

main().catch((err) => {
  console.error('[startup] failed:', err.message);
  process.exit(1);
});
```

15. React Login Component

client/src/pages/Auth.jsx

```
export function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const next = useNext();
  const [form, setForm] = useState({ email: '', password: '' });
  const [error, setError] = useState('');
  const [busy, setBusy] = useState(false);

  const submit = async (e) => {
    e.preventDefault();
    setBusy(true);
    setError('');
    try {
      await login(form.email, form.password);
      navigate(next, { replace: true });
    } catch (err) {
      setError(err.message);
    } finally {
      setBusy(false);
    }
  };

  return (
    <AuthFrame title="Welcome back" subtitle="Sign in to continue modernizing your
legacy code." footer={<>New here? <Link to="/register">Create an account</Link></>}>
      <form onSubmit={submit} className="form" noValidate>
        <Field label="Email" htmlFor="email">
          <input id="email" type="email" autoComplete="email" required
value={form.email} onChange={(e) => setForm({ ...form, email: e.target.value })} />
        </Field>
        <Field label="Password" htmlFor="password">
          <input id="password" type="password" autoComplete="current-password"
required value={form.password} onChange={(e) => setForm({ ...form, password:
e.target.value })} />
        </Field>
        {error && <div className="alert alert-danger">{error}</div>}
        <Button type="submit" loading={busy} className="btn-block">
          Sign in
        </Button>
      </form>
    </AuthFrame>
  );
}
```

client/src/lib/auth.jsx

```
const login = useCallback(async (email, password) => {
  const res = await api.post('/api/auth/login', { email, password });
  setUser(res.user);
  return res.user;
}, []);
```

16. Dashboard Logic

client/src/pages/Dashboard.jsx — fetching statistics and projects

```
export function Dashboard() {
  const { user } = useAuth();
  const meta = useMeta();
  const [stats, setStats] = useState(null);
  const [projects, setProjects] = useState(null);

  useEffect(() => {
    api.get('/api/stats').then(setStats).catch(() => setStats({}));
    api.get('/api/projects').then((r) => setProjects(r.projects)).catch(() =>
setProjects([]));
  }, []);
```

client/src/api.js — shared request helper

```
async function request(method, url, body) {
  const options = { method, credentials: 'same-origin', headers: { Accept:
'application/json' } };
  if (method !== 'GET') {
    options.headers['Content-Type'] = 'application/json';
    options.body = JSON.stringify(body ?? {});
  }
  let res;
  try {
    res = await fetch(url, options);
  } catch {
    throw new ApiError(0, 'Cannot reach the server. Check your connection and try
again.');
  }
  const type = res.headers.get('content-type') || '';
  const data = type.includes('application/json') ? await res.json().catch(() => ({}))
: await res.text();
  if (!res.ok) {
    const err = new ApiError(res.status, (data && data.error) || res.statusText ||
'Request failed', data?.details);
    if (res.status === 401 && unauthorizedHandler && !url.startsWith('/api/auth/'))
unauthorizedHandler(err);
    throw err;
  }
  return data;
}
```

17. REST API Request and Response Examples

The following requests and responses were captured from the running application.

Request — sign in

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{
  "email": "muthuveer.demo@legacylift.test",
  "password": "********"
}
```

Response

```
HTTP/1.1 200
Content-Type: application/json; charset=utf-8

```

```
{
  "user": {
    "id": "6ad10f999d509ca9086b7fd9",
    "name": "Muthuveer M",
    "email": "muthuveer.demo@legacylift.test",
    "createdAt": "2026-10-15T17:38:33.977Z"
  }
}
```

Request — create a project from a sample

```
POST /api/projects HTTP/1.1
Content-Type: application/json

{
  "name": "Weekly Payroll Modernization",
  "description": "Bundled cobol-payroll sample",
  "sampleId": "cobol-payroll"
}
```

Response

```
HTTP/1.1 201
Content-Type: application/json; charset=utf-8

{
  "project": {
    "id": "6ad10f9a9d509ca9086b7fda",
    "name": "Weekly Payroll Modernization",
    "description": "Bundled cobol-payroll sample",
    "target": {
      "language": "java",
      "framework": "plain"
    },
    "languages": [
      "cobol"
    ],
    "fileCount": 1,
    "lines": 90,
    "jobCount": 0,
    "lastJob": null,
    "merged": null,
    "createdAt": "2026-10-15T17:38:34.842Z",
    "updatedAt": "2026-10-15T17:38:34.842Z"
  }
}
```

Request — start a modernization job

```
POST /api/projects/6ad10f9a9d509ca9086b7fda/jobs HTTP/1.1
Content-Type: application/json

{
  "engine": "structural"
}
```

Response

```
HTTP/1.1 202
Content-Type: application/json; charset=utf-8

{
  "job": {
    "id": "6ad10f9a9d509ca9086b7fe4",
    "number": 1,
```

```
    "status": "queued",
    "engine": {
      "requested": "structural",
      "used": null
    },
    "target": {
      "language": "java",
      "framework": "plain"
    },
    "stages": {
      "parse": {
        "status": "pending"
      },
      "analyze": {
        "status": "pending"
      },
      "generate": {
        "status": "pending"
      },
      "validate": {
        "status": "pending"
      }
    },
    "score": null,
    "error": null,
    "parentJob": null,
    "reviewDecision": null,
    "fileCount": 0,
    "createdAt": "2026-10-15T17:38:34.930Z"
  }
}
```

Request — poll job status

```
GET /api/jobs/6ad10f9a9d509ca9086b7fe4/status HTTP/1.1
Content-Type: application/json
```

Response

```
HTTP/1.1 200
Content-Type: application/json; charset=utf-8

{
  "status": "review",
  "stages": {
    "parse": {
      "status": "done",
      "startedAt": "2026-10-15T17:38:34.964Z",
      "finishedAt": "2026-10-15T17:38:35.006Z",
      "message": "1 file(s), 1 module(s), 0 diagnostic(s)"
    },
    "analyze": {
      "status": "done",
      "startedAt": "2026-10-15T17:38:35.008Z",
      "finishedAt": "2026-10-15T17:38:35.013Z",
      "message": "6 procedure(s), max complexity 3, 1 finding(s)"
    },
    "generate": {
      "status": "done",
      "startedAt": "2026-10-15T17:38:35.020Z",
      "finishedAt": "2026-10-15T17:38:35.026Z",
      "message": "3 file(s) generated by the rule-based translator"
    },
    "validate": {
      "status": "done",
      "startedAt": "2026-10-15T17:38:35.031Z",
      "finishedAt": "2026-10-15T17:38:35.046Z",
      "message": "Fidelity score 100/100"
    }
```

```
  },
  "progress": {
    "stage": "review",
    "message": "Ready for human review"
  },
  "error": null,
  "score": 100
}
```

Request — approve a job

```
POST /api/jobs/6ad10f9a9d509ca9086b7fe4/review HTTP/1.1
Content-Type: application/json

{
  "decision": "approved",
  "summary": "Behaviour matches the legacy payroll output."
}
```

Response

```
HTTP/1.1 200
Content-Type: application/json; charset=utf-8

{
  "job": {
    "id": "6ad10f9a9d509ca9086b7fe4",
    "number": 1,
    "status": "approved",
    "review": {
      "decision": "approved",
      "summary": "Behaviour matches the legacy payroll output."
    }
  },
  "newJob": null
}
```

Request — merge an approved job

```
POST /api/jobs/6ad10f9a9d509ca9086b7fe4/merge HTTP/1.1
Content-Type: application/json
```

Response

```
HTTP/1.1 200
Content-Type: application/json; charset=utf-8

{
  "merged": {
    "jobId": "6ad10f9a9d509ca9086b7fe4",
    "jobNumber": 1,
    "mergedAt": "2026-10-15T17:39:01.872Z",
    "fileCount": 3
  }
}
```

Request — delete a project

```
DELETE /api/projects/6ad10fb89d509ca9086b7fff HTTP/1.1
Content-Type: application/json
```

Response

```
HTTP/1.1 200
Content-Type: application/json; charset=utf-8

{
  "ok": true
}
```

Request — without a session

```
GET /api/projects HTTP/1.1
Content-Type: application/json
# no session cookie
```

Response

```
HTTP/1.1 401
Content-Type: application/json; charset=utf-8

{
  "error": "Sign in to continue"
}
```

SCREENSHOTS OF THE PROJECT

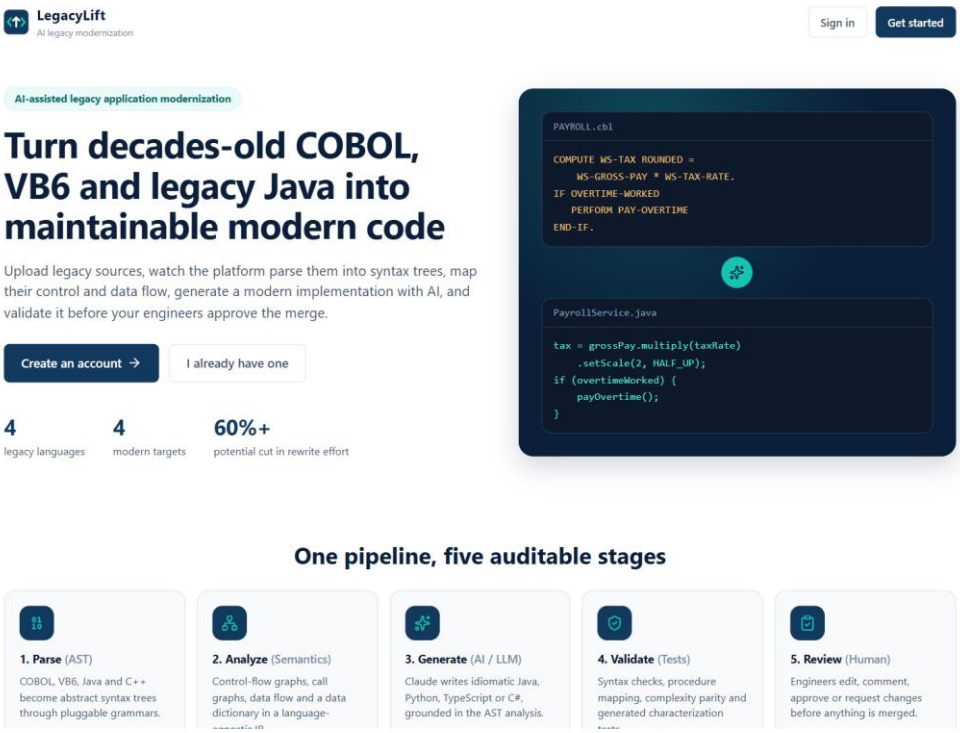


Figure A.1 Landing page

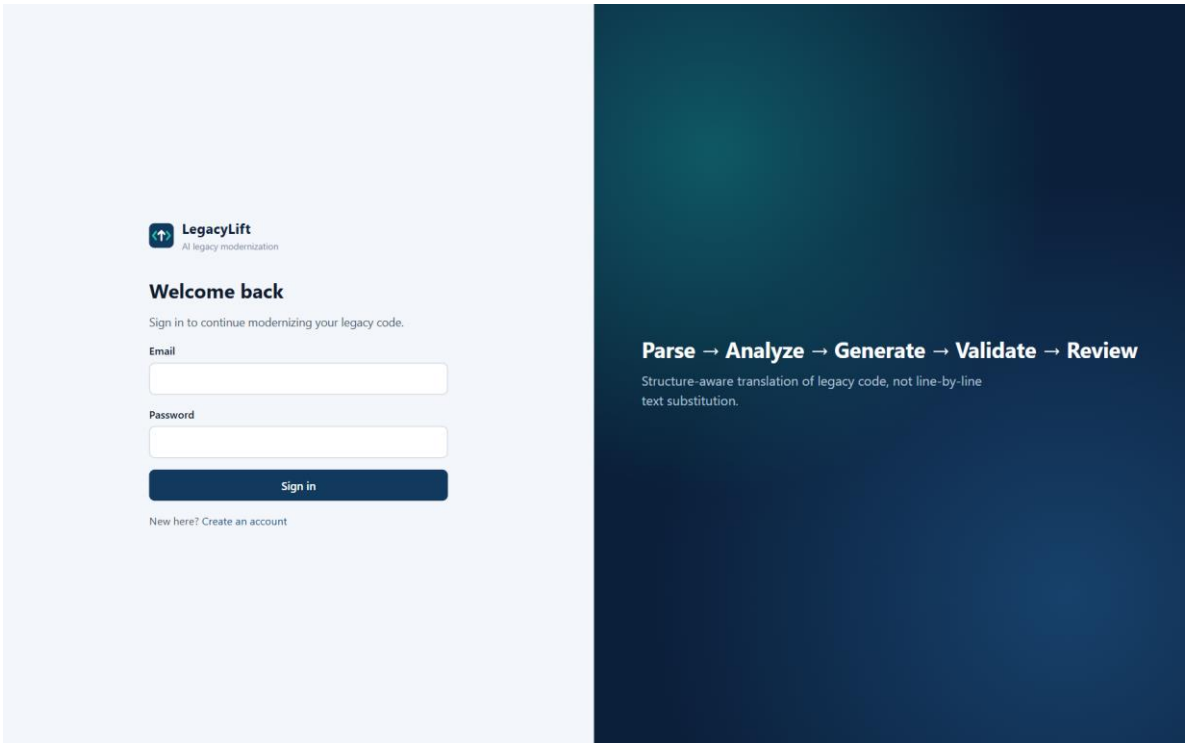


Figure A.2 Sign-in page

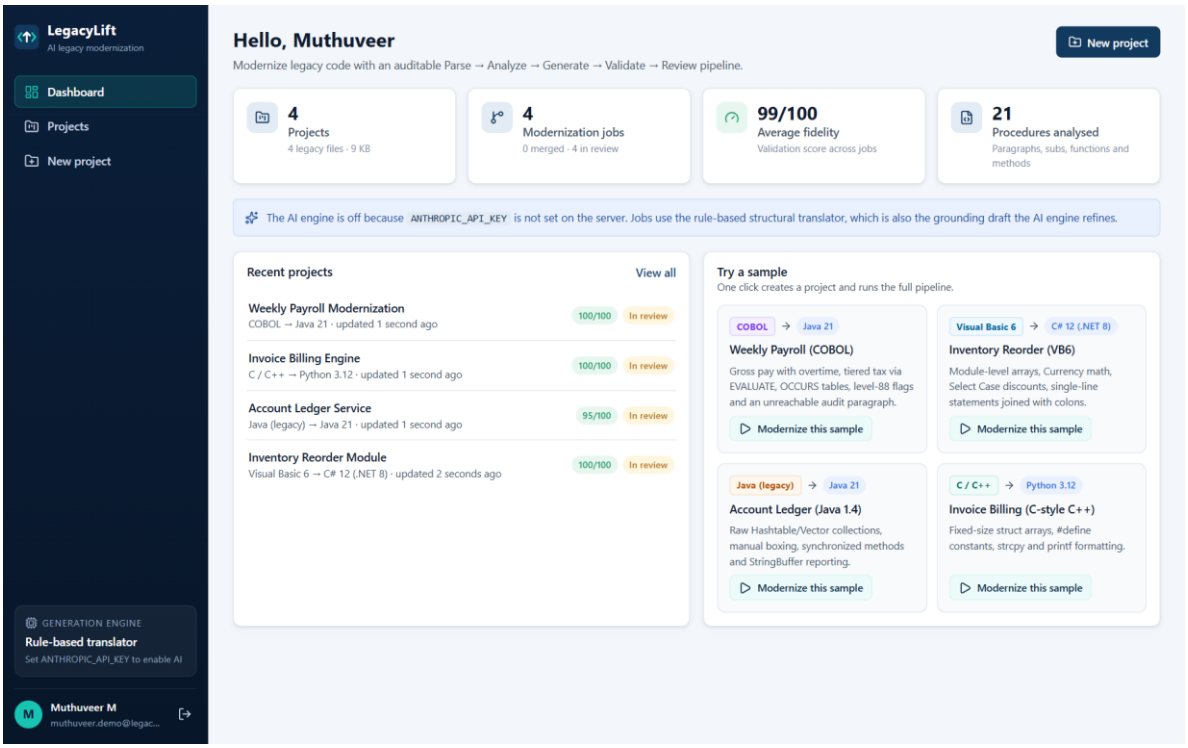


Figure A.3 Dashboard with project statistics and bundled samples

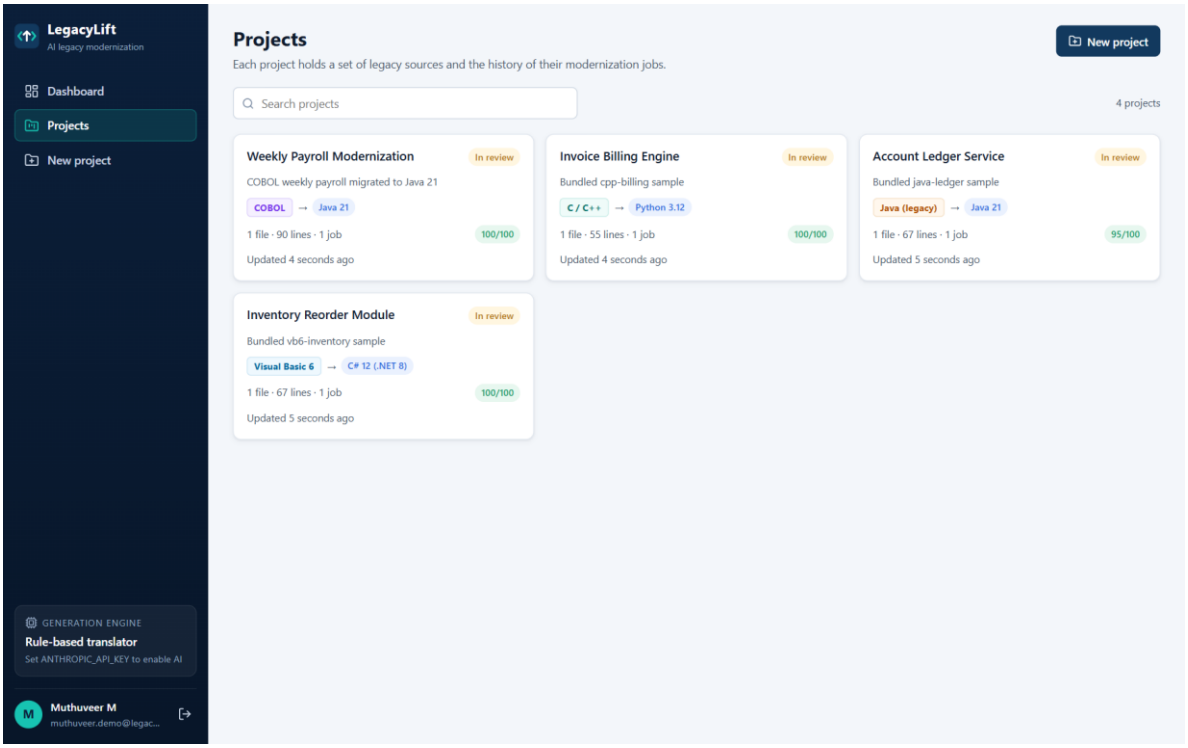


Figure A.4 Projects list

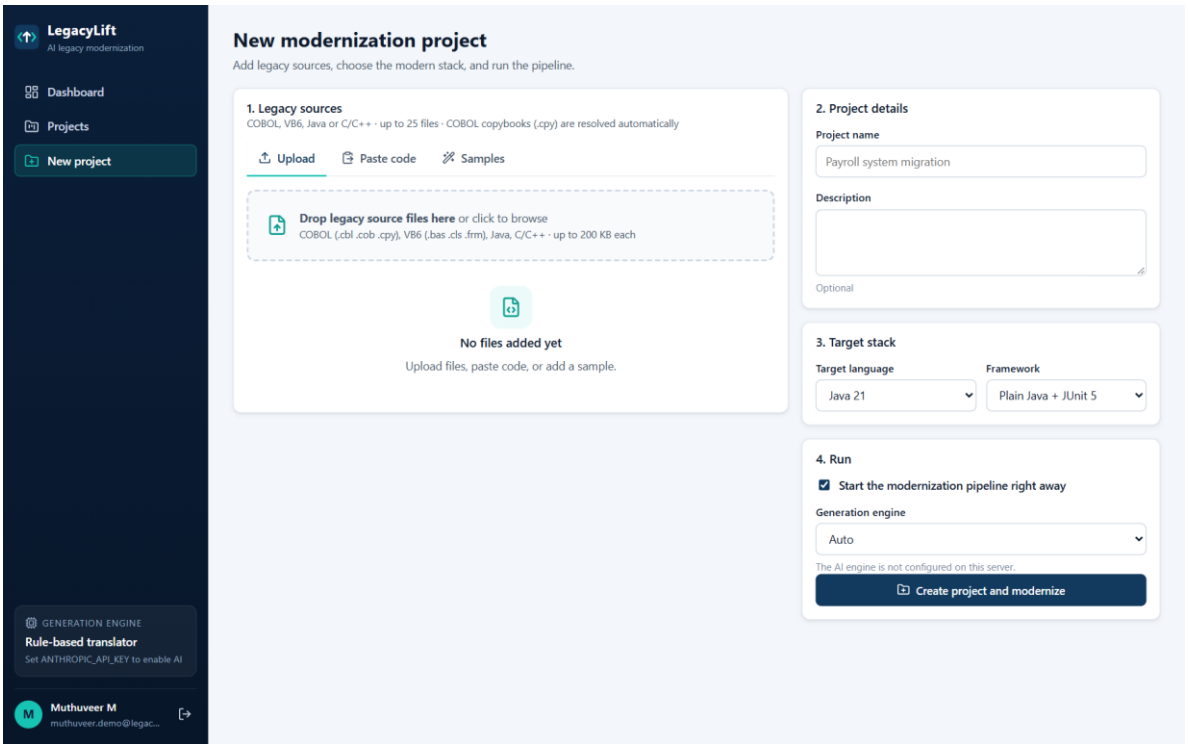


Figure A.5 Creating a new project: upload legacy files and choose the target

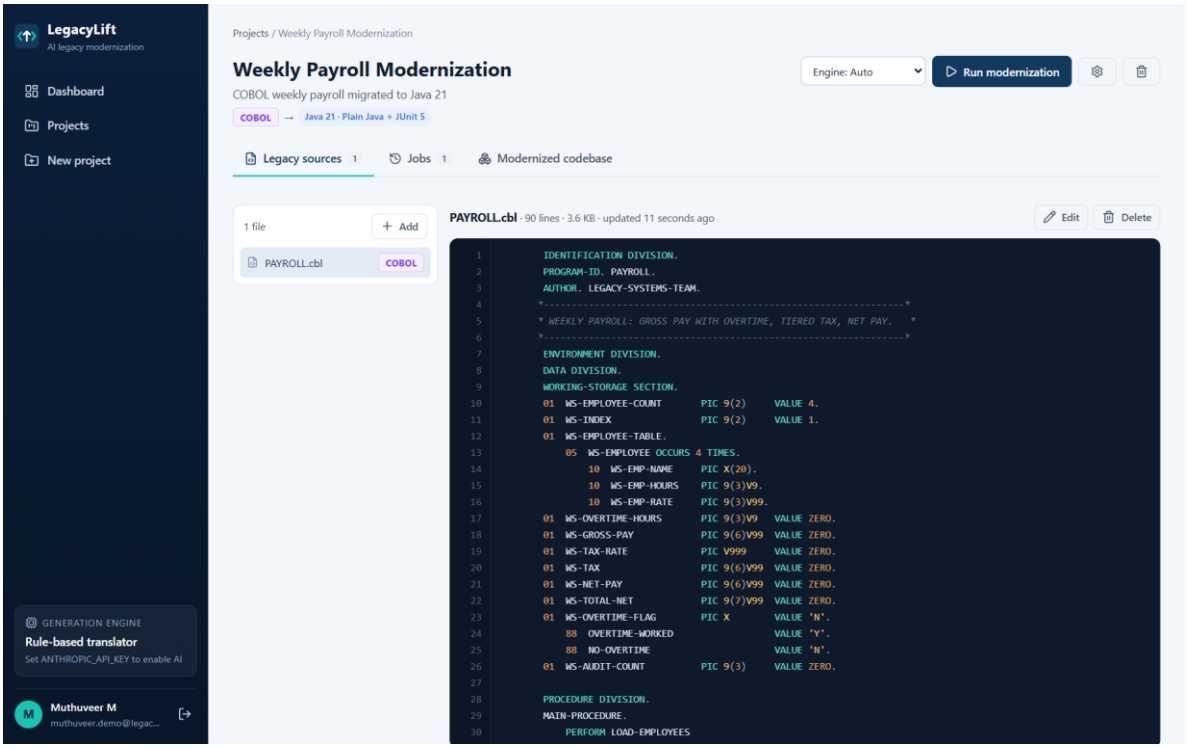


Figure A.6 Project detail with legacy sources and job history

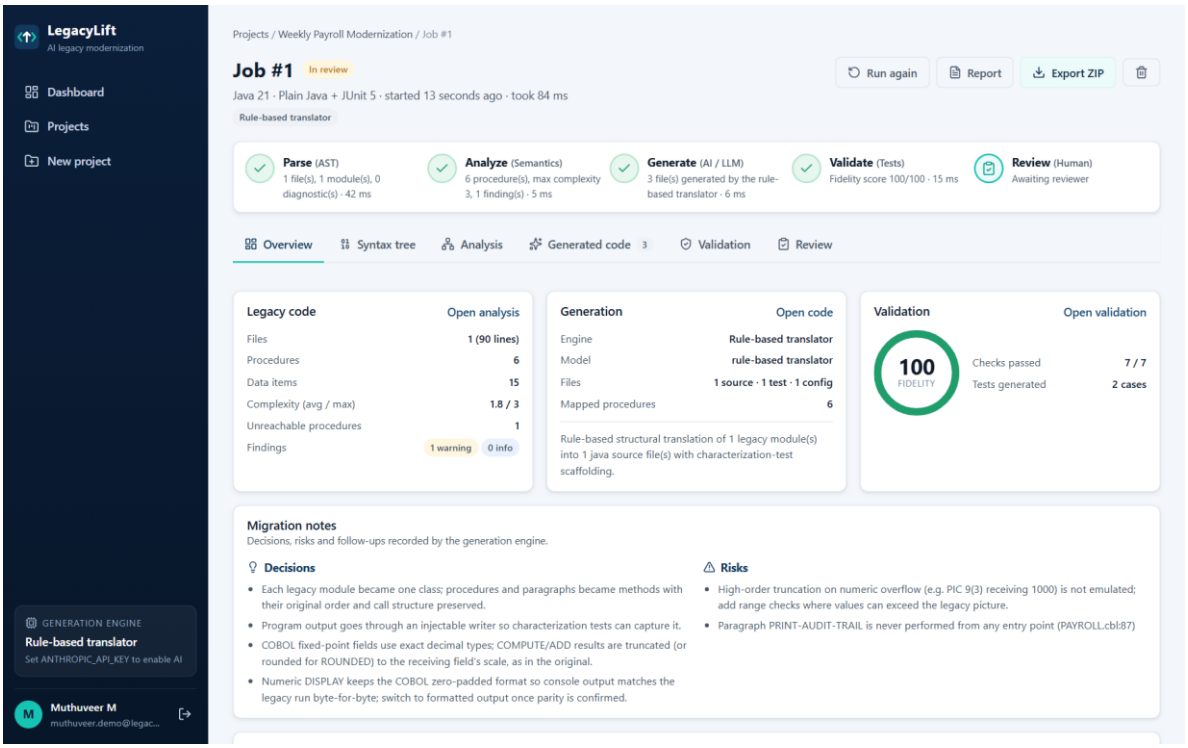


Figure A.7 Job workspace — overview of the pipeline result

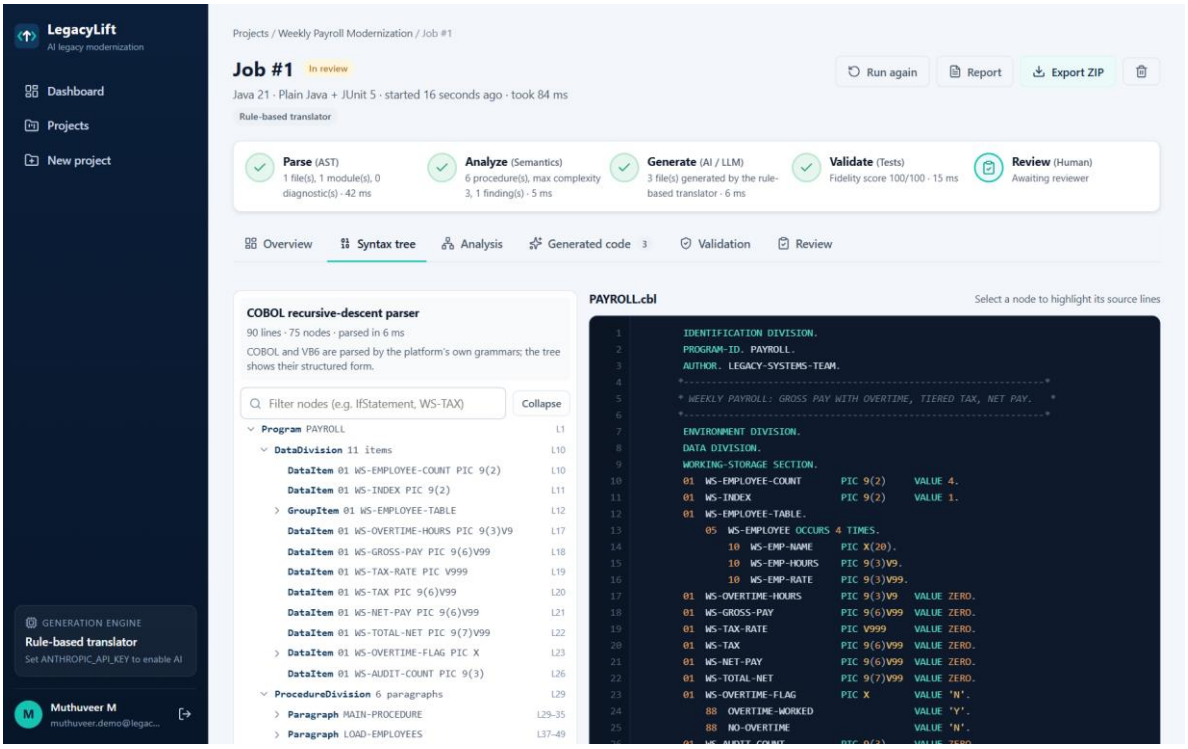


Figure A.8 Syntax tree explorer linked to the legacy source

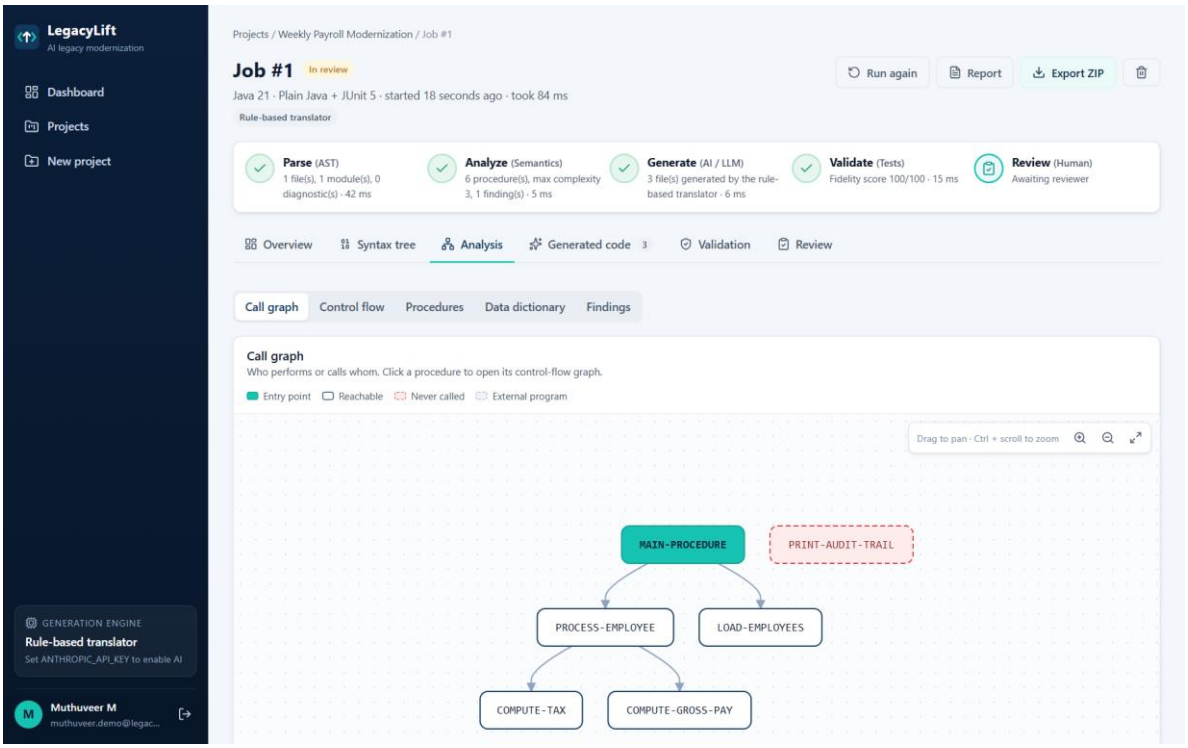


Figure A.9 Analysis — call graph with entry point and unreachable paragraph

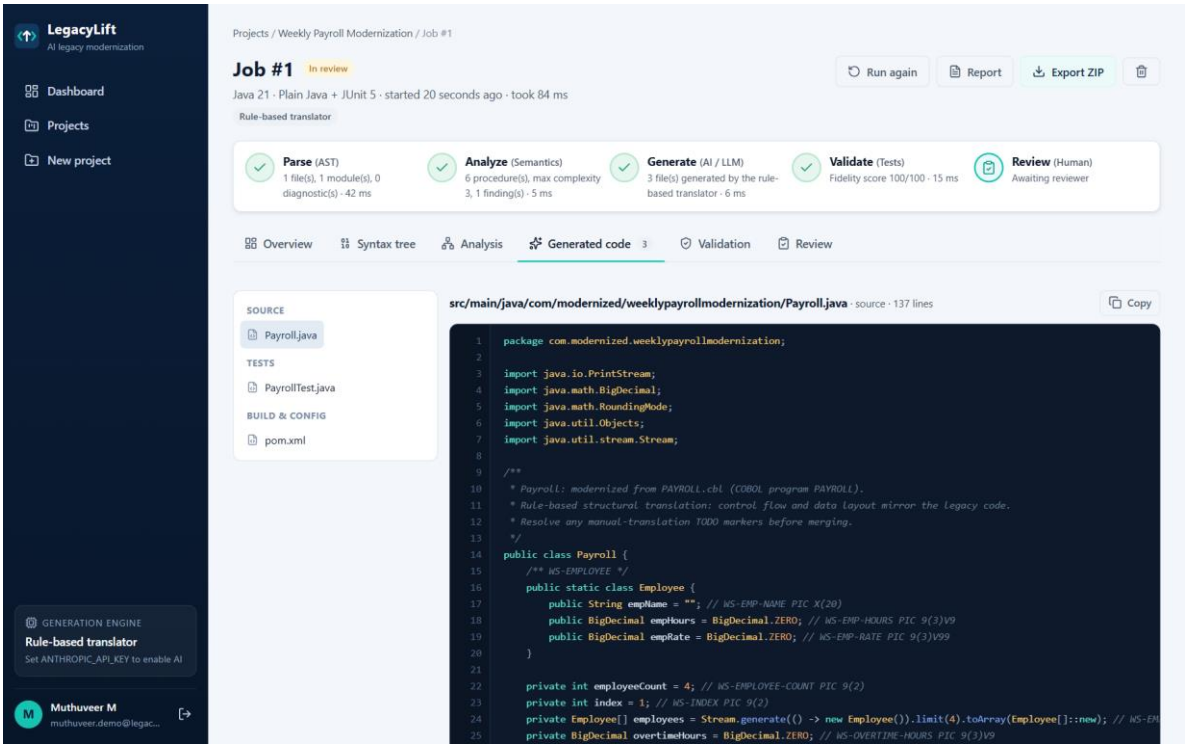


Figure A.10 Generated Java code with legacy-to-modern procedure mapping

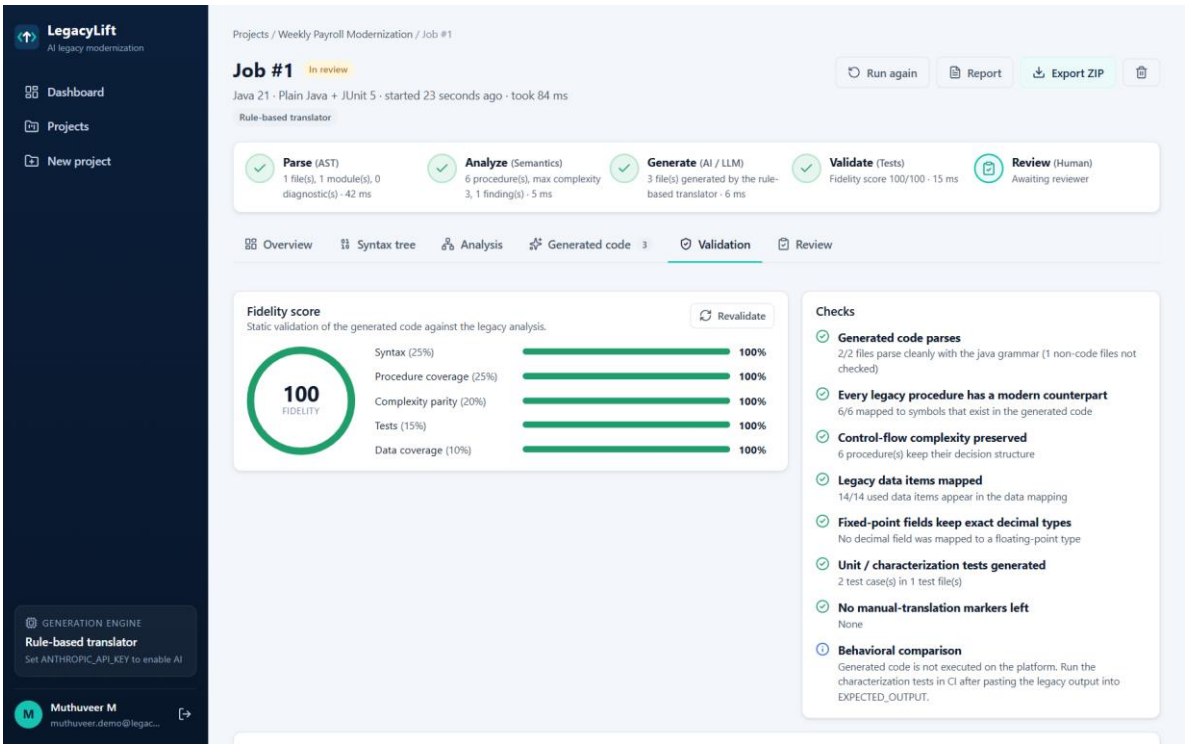


Figure A.11 Validation checks and fidelity score

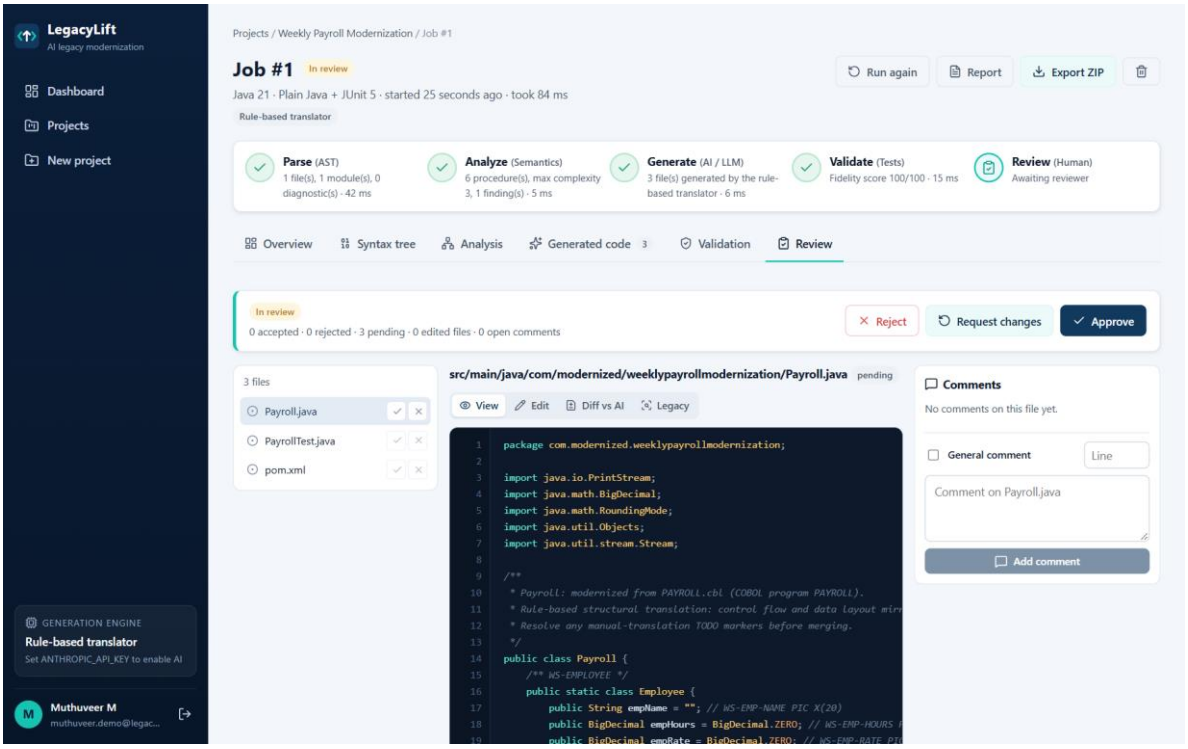


Figure A.12 Human review: file decisions, editor, diff and comments

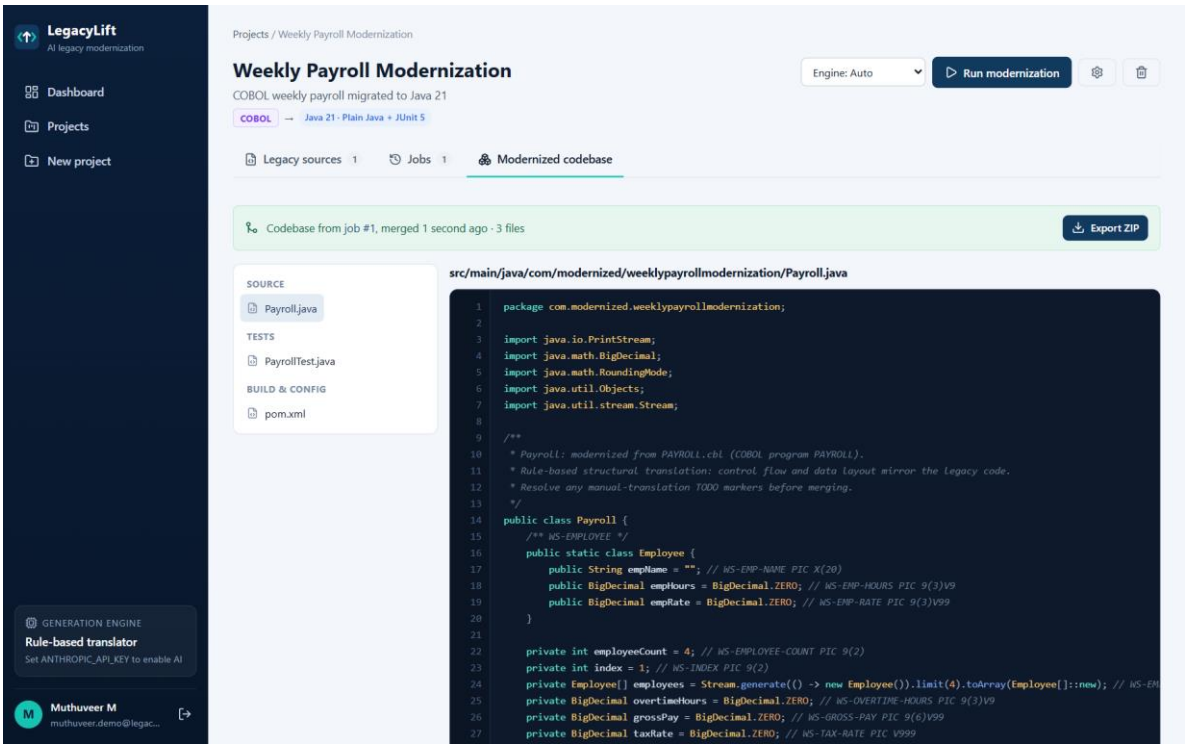


Figure A.13 Modernized codebase after the approved job has been merged

REFERENCES

[1] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.

[2] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, 2020.

[3] T. Parr, *The Definitive ANTLR 4 Reference*. Raleigh, NC, USA: Pragmatic Bookshelf, 2013.

[4] R. C. Seacord, D. Plakosh and G. A. Lewis, *Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices*. Boston, MA, USA: Addison-Wesley, 2003.

[5] Tree-sitter, “An Incremental Parsing System for Programming Tools,” documentation. [Online]. Available: https://tree-sitter.github.io/tree-sitter/

[6] Anthropic, “Claude API Documentation: Messages, Streaming and Structured Outputs.” [Online]. Available: https://docs.anthropic.com/

[7] MongoDB, Inc., “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/

[8] Mongoose, “Mongoose ODM Documentation.” [Online]. Available: https://mongoosejs.com/docs/

[9] OpenJS Foundation, “Express 5 API Reference.” [Online]. Available: https://expressjs.com/

[10] OpenJS Foundation, “Node.js 22 Documentation.” [Online]. Available: https://nodejs.org/docs/

[11] Meta Open Source, “React 19 Documentation.” [Online]. Available: https://react.dev/

[12] T. J. McCabe, “A Complexity Measure,” *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.

[13] OWASP Foundation, “OWASP Top Ten Web Application Security Risks.” [Online]. Available: https://owasp.org/www-project-top-ten/